Municipal Services Application

Project Completion Report

Part 3 - Full-Functioning Application

Student Number: ST10202185
Module: AAPD7112 - Advanced Application Development
Technology: C# .NET Framework 4.7.2 (Windows Forms)

1. Introduction

This report documents the completion of the Municipal Services Application, a C# .NET Framework Windows Forms project developed for the City of Ekurhuleni, Benoni District. The application was built incrementally across three parts. Part 1 delivered the Report Issues feature, allowing residents to log municipal problems. Part 2 added the Local Events and Announcements module, built on a range of intermediate data structures. Part 3, covered in detail by this report, completes the application by adding the Service Request Status feature, which allows residents to track submitted requests and which is powered by an extensive set of trees, heaps, and graph algorithms.

The aim throughout has been to create a single, user-centric platform that improves citizen engagement and service delivery, while demonstrating the correct and efficient use of advanced data structures and algorithms learned across the module.

2. Project Overview

The finished application opens on a main menu offering three features, all now fully functional:

Feature	Part	Summary
Report Issues	Part 1	Residents log issues with location, category, description, and file attachments.
Local Events & Announcements	Part 2	Residents browse events and notices, search and filter them, and receive recommendations.
Service Request Status	Part 3	Residents submit and track service requests, organised and retrieved using trees, heaps, and a graph.

Part 3 focused on the final feature and on integrating advanced data structures. The Service Request Status page presents requests in six tabs: All Requests, By Priority, Tree Explorer, By Ward / Graph, Search by ID, and Submit New.

3. Screenshots of the Running Application

The screenshots below were captured while running the completed application and illustrate, screen by screen, the full feature set delivered across all three parts of the project, from the initial Report Issues form through to the final Admin area.

3.1 Main Menu

The main menu provides access to every feature. All four options shown are fully functional: Report Issues (Part 1), Local Events and Announcements (Part 2), Service Request Status (Part 3), and the staff Admin Login added afterwards. A credentials banner at the bottom of the page is included for marking purposes.

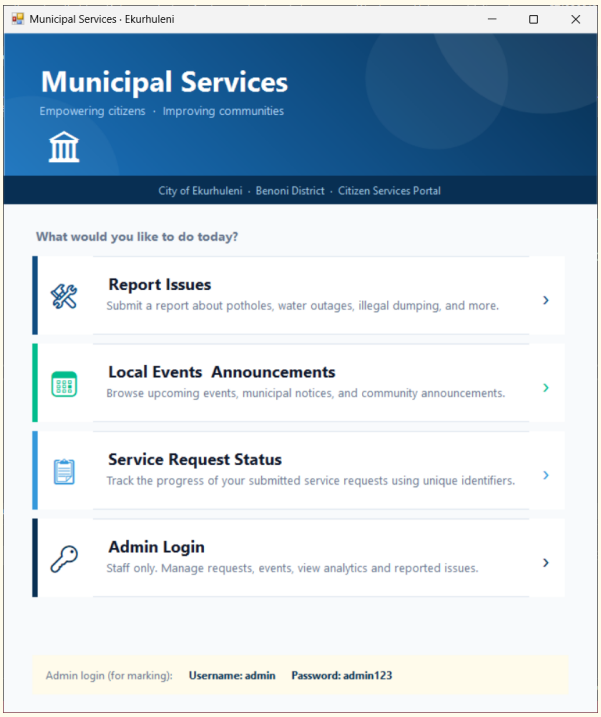


Figure 1 - Main menu with all features active

3.2 Part 1 - Report Issues

Residents use this form to report a municipal issue, including a location, category, description, and optional file attachments, with a progress indicator showing form completion.

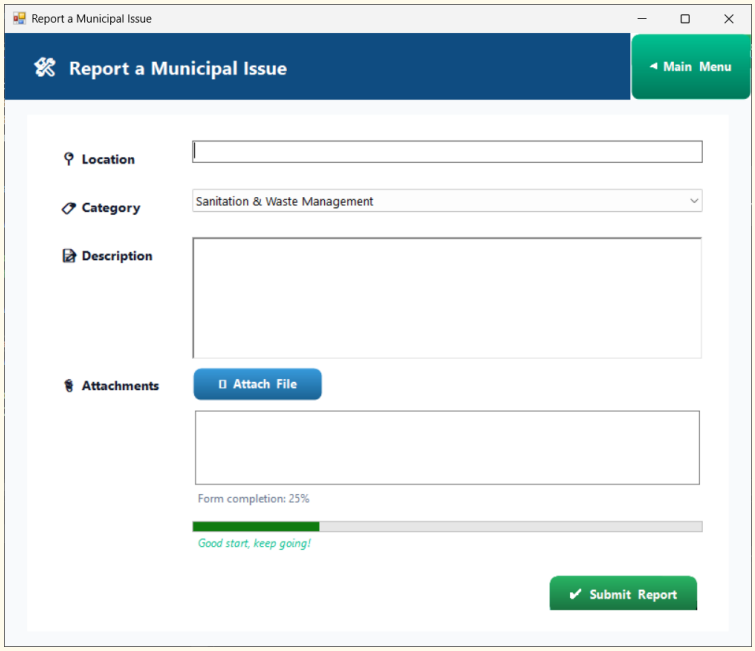


Figure 2 - Report Issues form, empty / initial state

As the resident fills in each field, the progress bar and engagement message update to reflect how complete the form is.

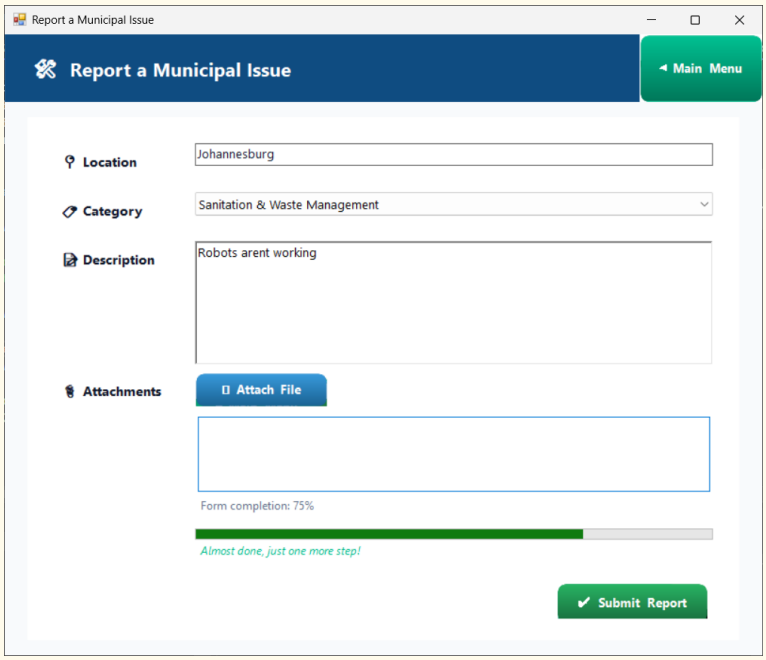


Figure 3 - Report Issues form, partially completed with progress bar

Selecting Attach File opens a file dialog so the resident can attach supporting images or documents to the report.

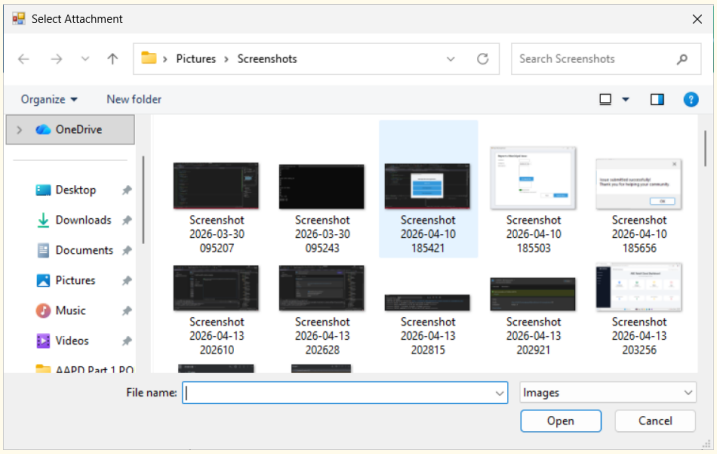


Figure 4 - Attaching a file to a report

On successful submission, a confirmation message is shown and the form resets, ready for the next report.

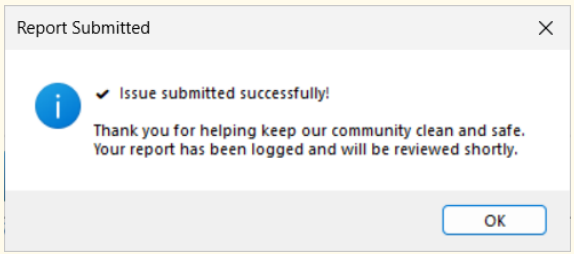


Figure 5 - Report Issues: submission confirmation

3.3 Part 2 - Local Events and Announcements

All municipal events and announcements are listed and sorted by date. Announcements are colour-coded, and the page is organised into tabs for All Events, Announcements, Urgent, Recommended, and Recently Viewed.

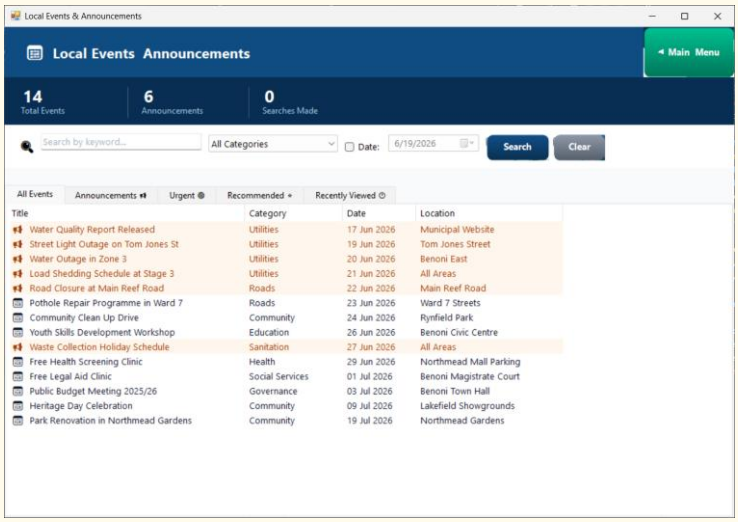


Figure 6 - Local Events and Announcements: full event listing (All Events tab)

The Announcements tab shows only municipal notices, in the order they were issued, powered by a FIFO queue.

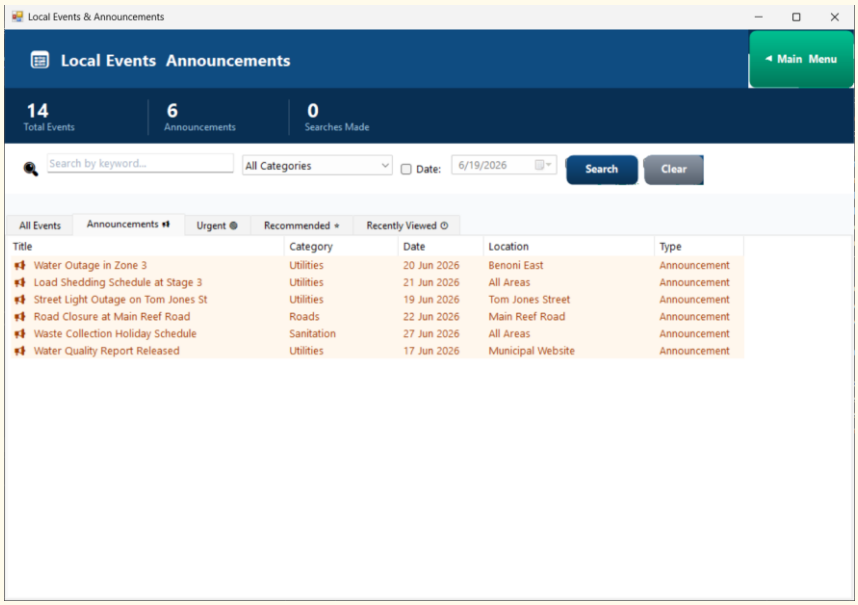


Figure 7 - Announcements tab

The Urgent tab surfaces the highest-priority events and announcements first, powered by a priority queue.

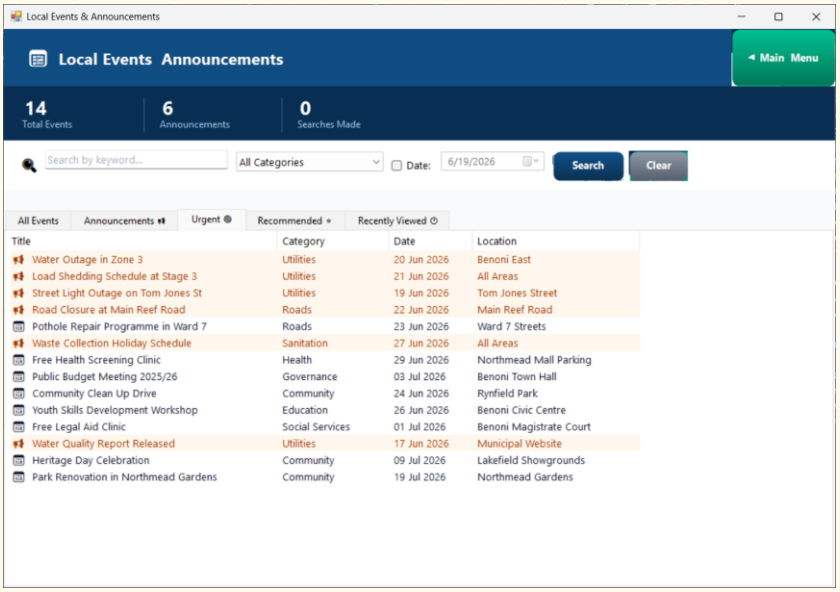


Figure 8 - Urgent tab

The search feature filters events by keyword, category, and date. Each search is recorded and used to power the Recommended tab.

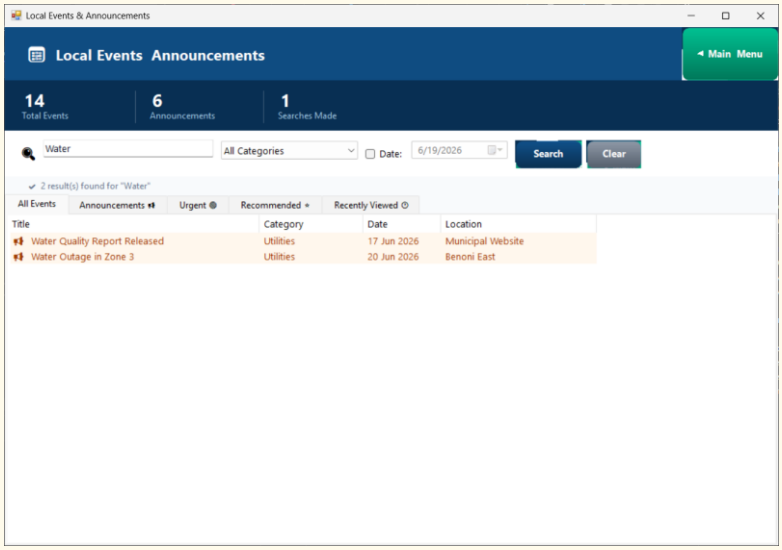


Figure 9 - Searching events by keyword

The Recommended tab analyses the resident's recent searches and suggests events likely to be of interest.

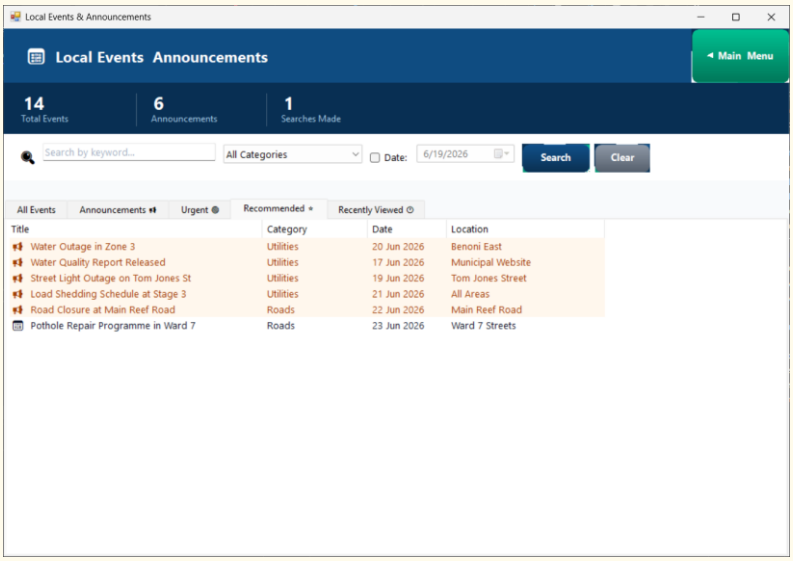


Figure 10 - Recommended tab

The Recently Viewed tab shows the events the resident has opened most recently, with the latest at the top, powered by a stack.

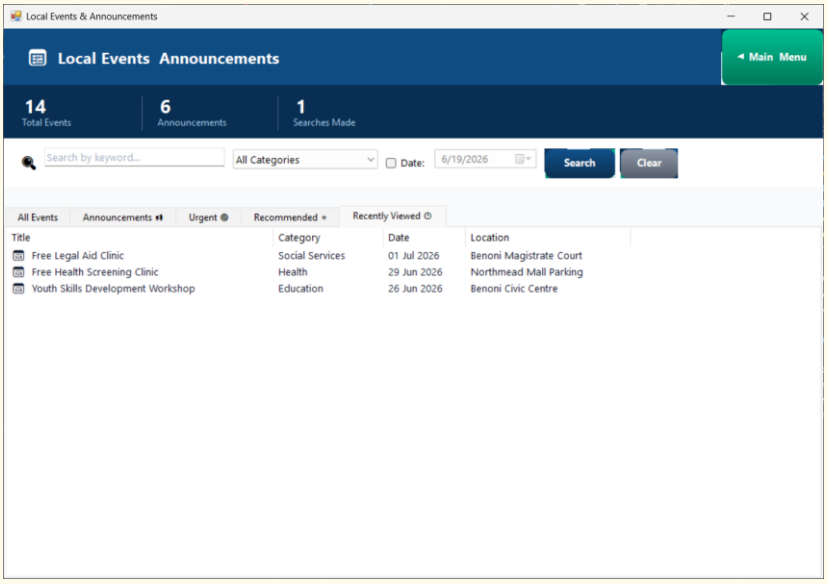


Figure 11 - Recently Viewed tab

Double-clicking any event opens a detail popup showing its full description, location, date, and category.

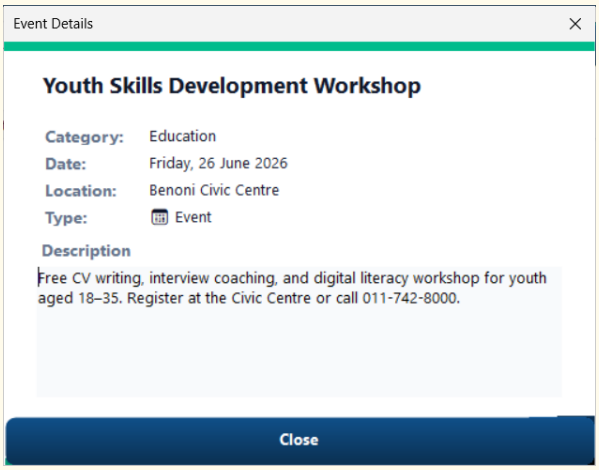


Figure 12 - Event detail popup

3.4 Part 3 - Service Request Status

The All Requests tab lists every service request sorted by ID, with filters for status and category.

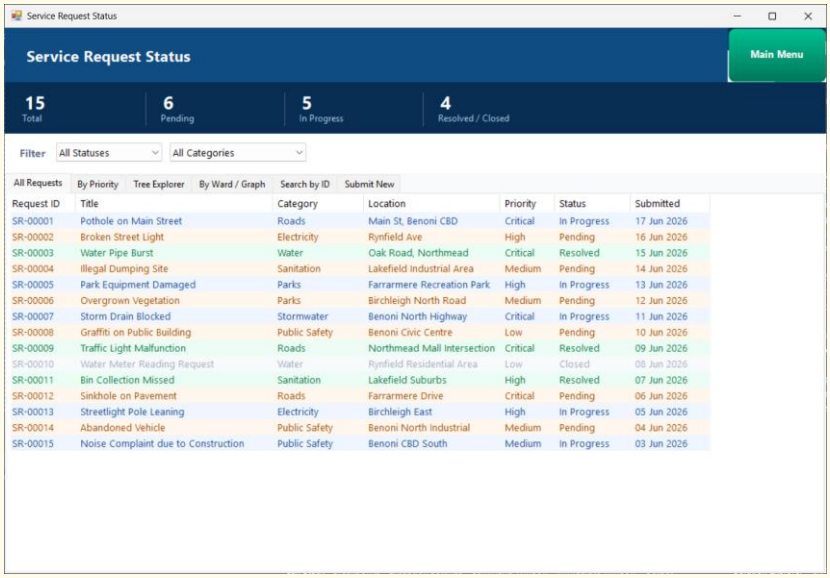


Figure 13 - Service Request Status: All Requests tab

The By Priority tab orders requests with the most urgent first, powered by a min-heap.

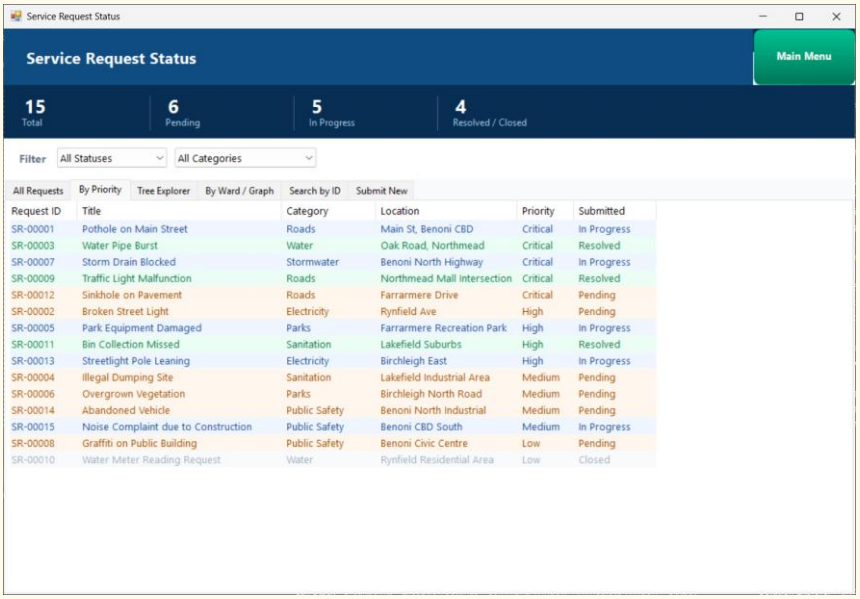


Figure 14 - By Priority tab

The Tree Explorer tab lets a user select any of the implemented tree structures and view its contents. The example below shows the Red-Black tree, with its in-order traversal correctly returning all 15 seeded requests sorted by ID.

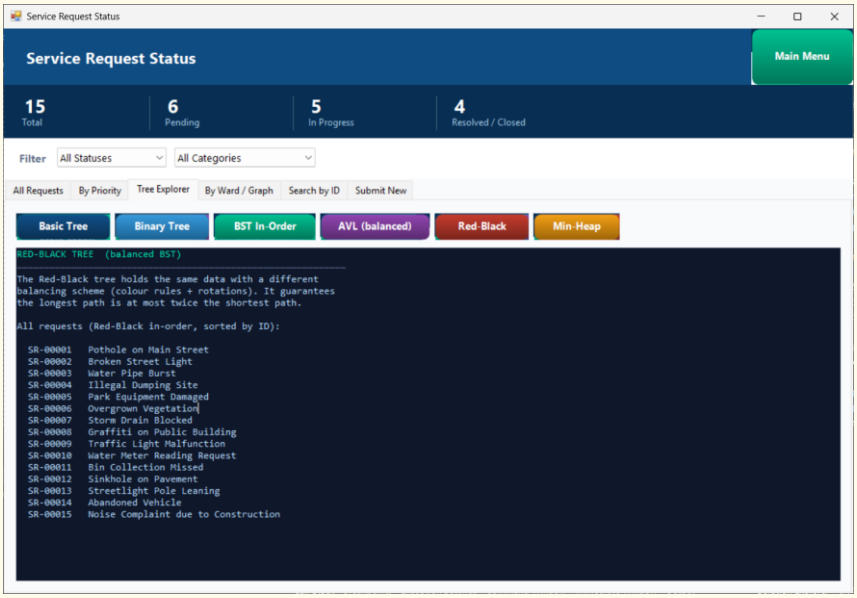


Figure 15 - Tree Explorer: Red-Black tree in-order traversal

The same Tree Explorer tab can also display the Basic Tree (department hierarchy), the plain Binary Tree (priority classification), the Binary Search Tree, the AVL tree, and the Min-Heap, on request via the corresponding buttons.

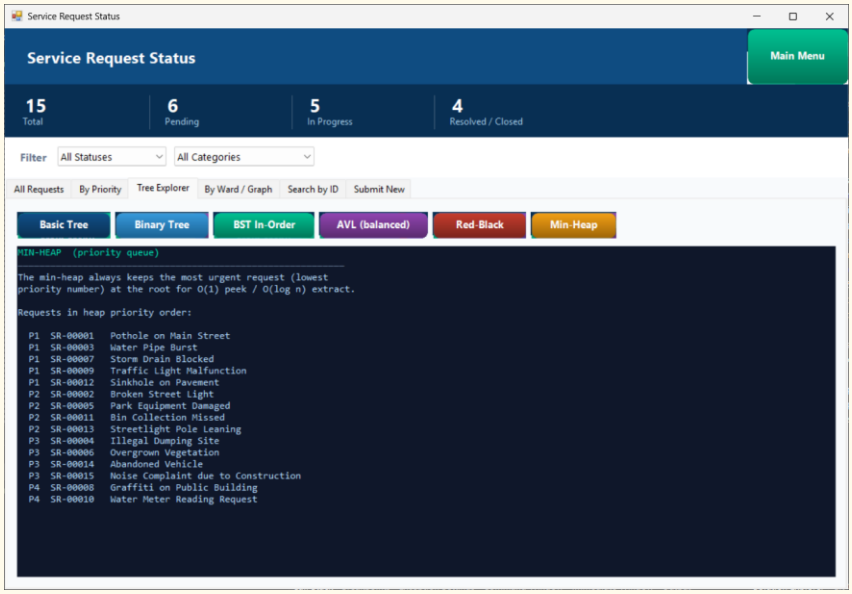


Figure 16 - Tree Explorer:Min-Heap output

The By Ward / Graph tab models the seven Ekurhuleni wards as a graph. The screenshot below shows the result of running Prim's Minimum Spanning Tree algorithm, which found the lowest-cost set of roads (30 km total) connecting all seven wards.

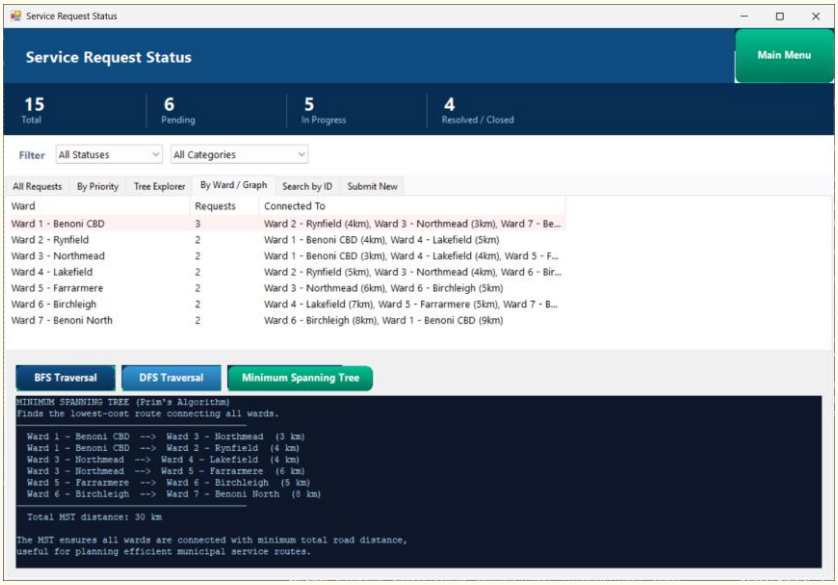


Figure 17 - By Ward / Graph: Minimum Spanning Tree result

The same tab can also run a Breadth-First Search or Depth-First Search starting from a chosen ward, printing the order in which wards are visited.

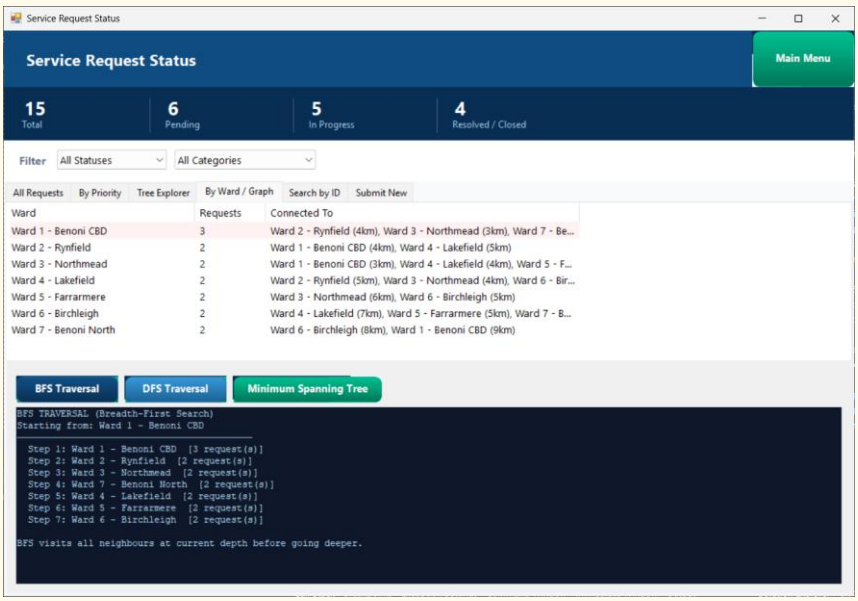


Figure 18 - By Ward / Graph: BFS traversal output

The Search by ID tab uses the AVL tree to fetch a specific request in guaranteed O(log n) time and allows its status to be updated.

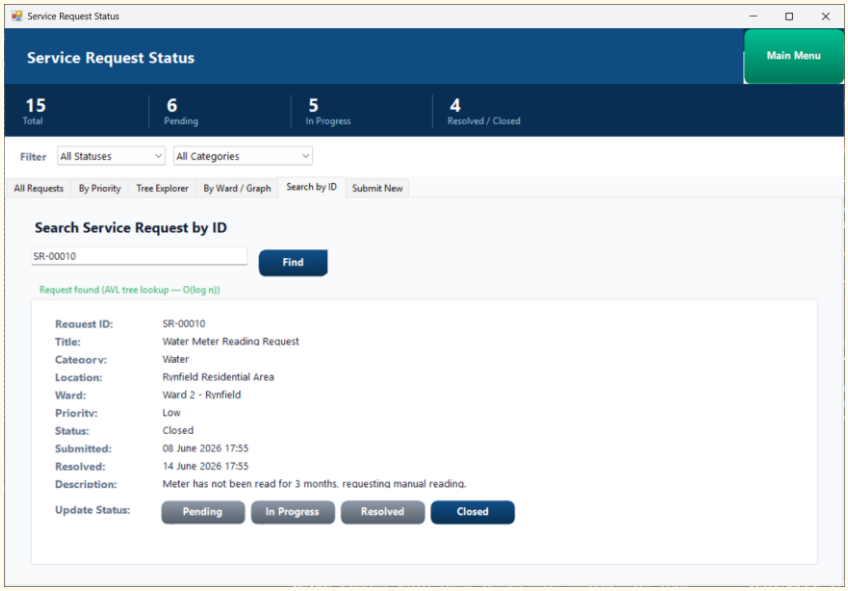


Figure 19 - Search by ID showing a found request

The Submit New tab allows a resident to lodge a new service request, which is inserted into every underlying data structure (the trees, the heap, and the ward graph) at the same time.

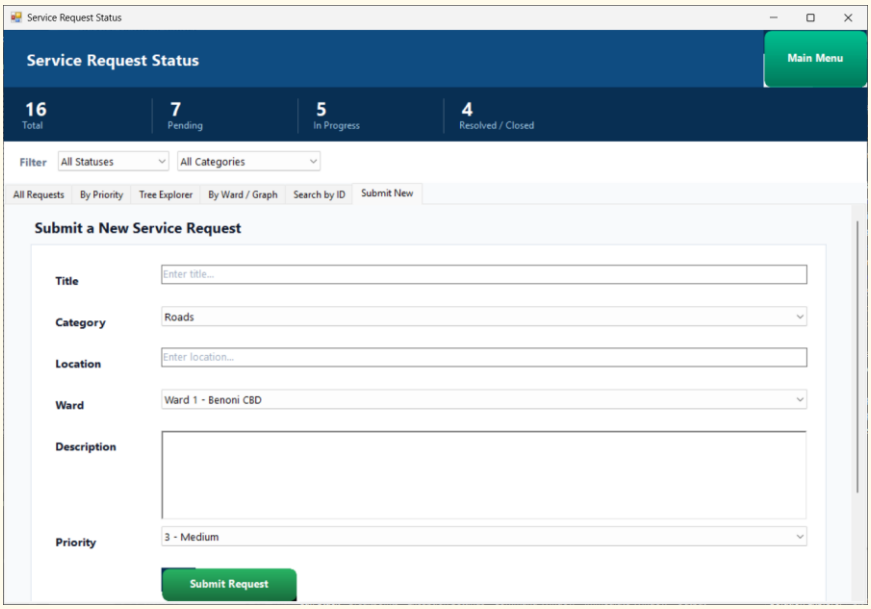


Figure 20 - Submit New service request form

On successful submission, the resident is shown their unique request ID, which they can use later in the Search by ID tab to track progress.

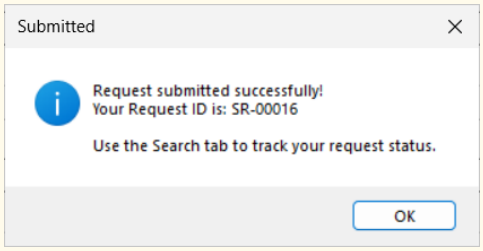


Figure 21 - Submit New: confirmation showing the generated request ID

3.5 Admin Area

A staff-only Admin area was added after the initial Part 3 submission. Access is protected by a username and password login.

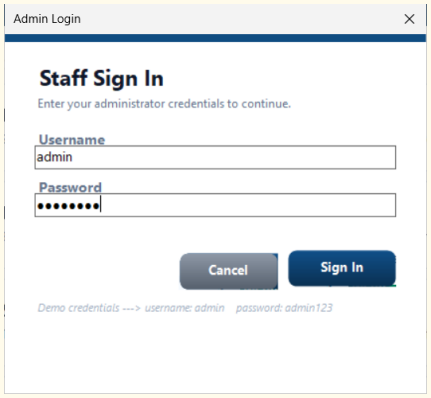


Figure 22 - Admin login screen

After signing in, staff see a live dashboard with statistics for requests, events, announcements, and reported issues, along with a bar chart of requests by status.

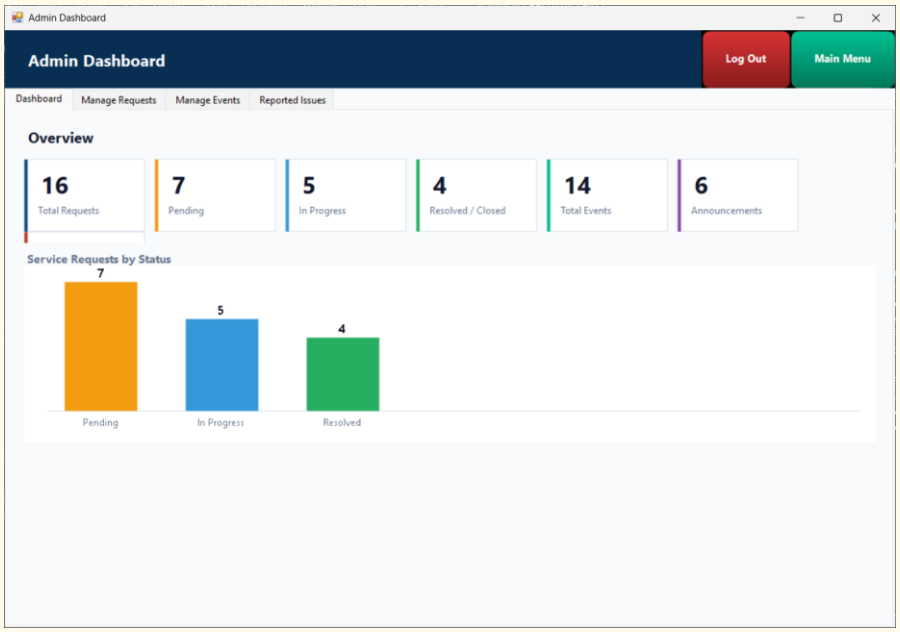


Figure 23 - Admin dashboard with statistics and chart

The Manage Requests tab lets staff update a request's status, reassign it to a different ward, or approve / reject it.

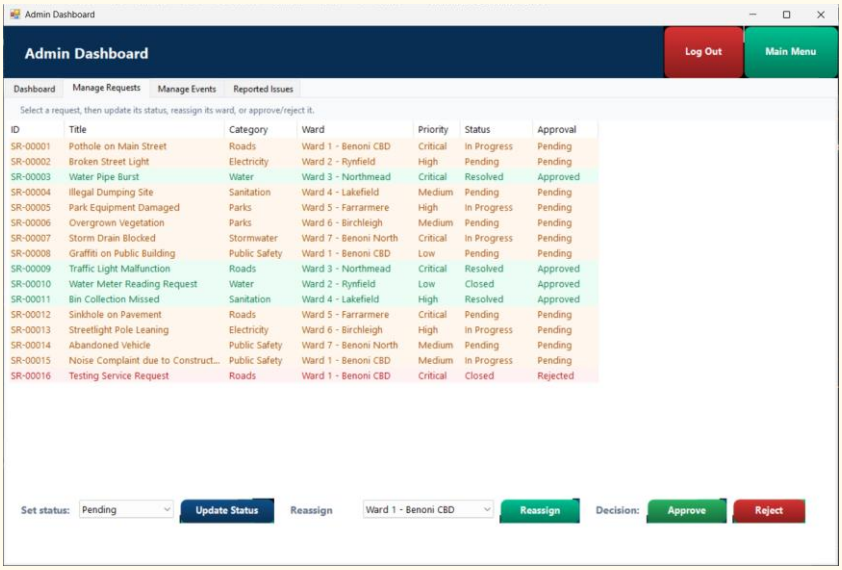


Figure 24 - Admin: managing service requests with approve/reject

The Manage Events tab lets staff add a new event or announcement, edit an existing one, or delete it.

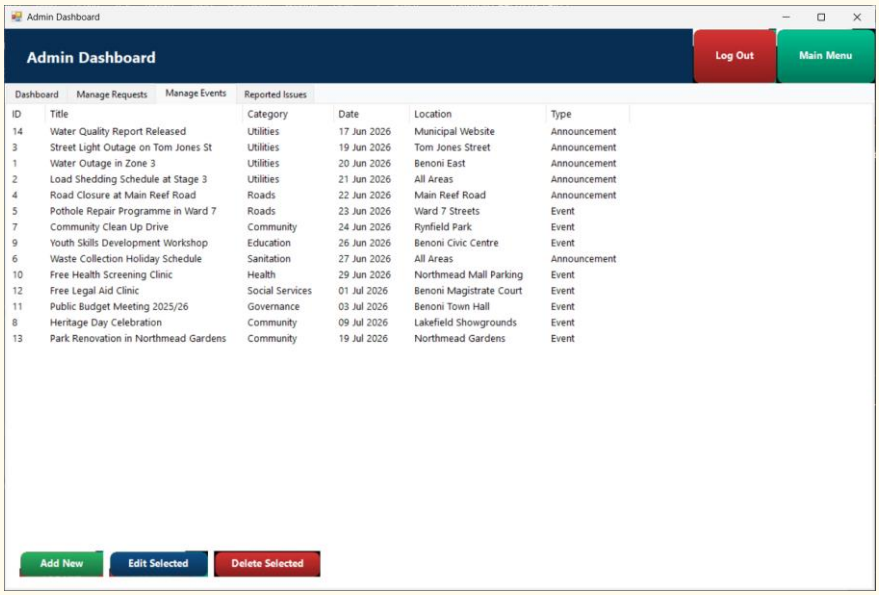


Figure 25 - Admin: managing events (add/edit/delete)

The Reported Issues tab lists every issue submitted by citizens through the Part 1 Report Issues feature, with the same approve / reject workflow.

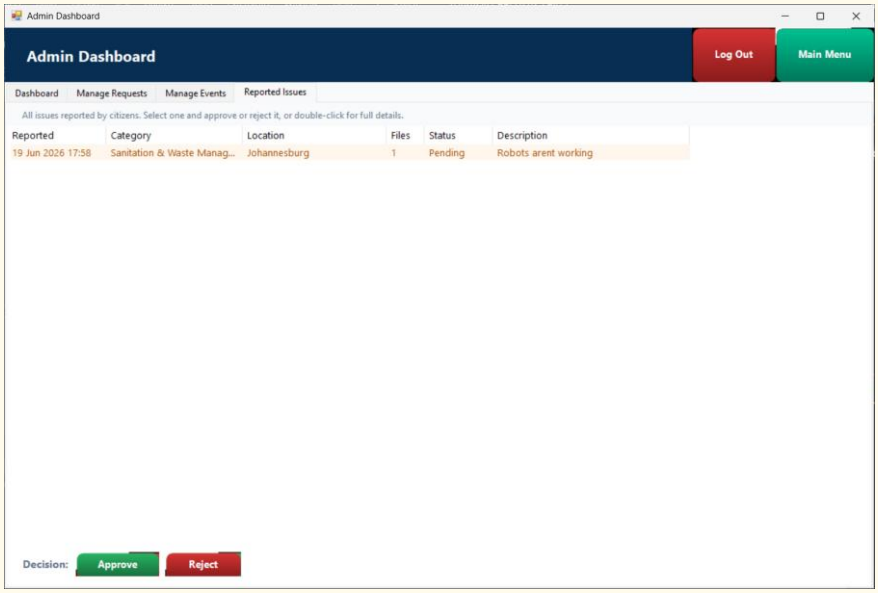


Figure 26 - Admin: reported issues with approve/reject

4. Task 3 Implementation

The Service Request Status feature stores each request in several data structures simultaneously, with each structure optimized for a particular operation. The following structures were implemented from scratch, without relying on any built-in tree or graph library.

4.1 Trees

Five distinct tree structures were implemented, covering basic trees, binary trees, binary search trees, AVL trees, and red-black trees.

- Basic Tree: a generic hierarchy modelling departments and the categories beneath them. It is used to route a request to the correct municipal department.
- Binary Tree: a fixed-shape decision tree that classifies a request's numeric priority into a readable label such as Critical or Low. It demonstrates a binary tree used for classification rather than searching.
- Binary Search Tree (BST): stores requests by ID; in-order traversal returns them sorted, powering the All Requests tab. Average O(log n) insertion and search.
- AVL Tree: a self-balancing BST used for the Search by ID feature, guaranteeing O(log n) lookups even in the worst case through rotations.
- Red-Black Tree: a second balanced BST using colour rules and rotations, demonstrating an alternative balancing strategy on the same data.

4.2 Heaps, Graphs, Graph Traversal, and Minimum Spanning Tree

- Min-Heap: a priority queue that keeps the most urgent request at the root. Peek is O(1) and extract-minimum is O(log n). It powers the By Priority tab.
- Graph: an adjacency-list graph modelling the seven Ekurhuleni wards as vertices and the roads between them as weighted edges, with each ward holding its own service requests.
- Breadth-First Search (BFS): visits wards layer by layer from a starting ward, useful for finding nearby wards first. O(V + E).
- Depth-First Search (DFS): explores each branch fully before backtracking, useful for reachability. O(V + E).
- Minimum Spanning Tree (Prim's algorithm): finds the lowest-cost set of roads connecting every ward, directly applicable to planning efficient maintenance routes.

5. Challenges Faced and How They Were Overcome

Several challenges arose during Part 3. The most significant are described below along with the solutions applied.

Challenge 1: Windows Forms layout and overlapping controls

Early versions of the menu and the events page suffered from overlapping controls and clipped text. The cause was the order in which docked controls were added to their parent: in Windows Forms, a Fill-docked control must be added before Top, Bottom, Left, and Right controls, otherwise it consumes space the others need. The solution was to add controls in the correct reverse-dock order, and to

move custom-painted rounded panels out of the docking path by giving them a plain inner panel to hold the docked children. This removed all overlap and clipping.

Challenge 2: Search results counting but not displaying

On the events page, a search would report the correct number of matches but show an empty list. The root cause was again dock ordering: the results TabControl was added to the form after the header panels, so it had no room to render. Adding the Fill control first resolved it. This experience directly informed how the Service Request Status form was built, so the same issue did not recur.

Challenge 3: Keeping a plain Binary Tree distinct from the BST

The brief lists basic trees and binary trees separately from binary search trees. It would have been easy to treat them as the same thing. To keep them genuinely distinct, the plain binary tree was implemented as a fixed-shape classification tree that never searches, while the BST, AVL, and red-black trees handle ordered storage and lookup. This makes the difference between a binary tree and a binary search tree explicit in the code.

Challenge 4: Guaranteeing fast lookups regardless of insertion order

Service request IDs are generated in sequence, so inserting them into a plain BST would create an unbalanced, list-like tree with slow O(n) lookups. The AVL tree was introduced specifically for the Search by ID feature so that lookups remain O(log n) no matter the insertion order, by rebalancing with rotations after each insert.

Challenge 5: Building the application incrementally without breaking earlier parts

Because each part builds on the previous one, there was a risk of breaking Part 1 or Part 2 while adding Part 3. This was managed by keeping each feature in its own form and data-store class, sharing only a common theme and a set of custom controls. New structures for Part 3 were placed in a dedicated file, so the earlier code remained untouched and continued to work.

6. Key Learnings

The project produced several important learnings across technical skills, problem-solving, and programming technique.

- Choosing the right data structure matters more than raw code. Each structure in Part 3 was selected because it makes one operation cheap: a heap for the most-urgent request, an AVL tree for guaranteed-fast lookups, a graph for routing. Matching the structure to the operation is the core skill.
- Self-balancing trees solve a real problem. Seeing how sequential IDs degrade a plain BST, and how the AVL tree prevents this, made the value of balancing concrete rather than theoretical.
- Understanding the framework prevents bugs. The dock-ordering issues were not logic errors but framework behaviour. Learning how Windows Forms lays out docked controls removed an entire class of bugs.

- Separating concerns keeps a growing project stable. Keeping data structures, models, and UI in separate files allowed the project to triple in size across three parts without earlier features breaking.
- Implementing algorithms from scratch builds real understanding. Writing the rotations for the AVL and red-black trees, the sift operations for the heap, and Prim's algorithm for the MST gave a much deeper understanding than calling a library would have.

7. Technology Recommendations

The following technologies and tools could enhance the functionality, performance, and maintainability of the Municipal Services Application in future. Each recommendation includes a justification based on the project's needs and its compatibility with a C# code base.

7.1 A database (SQLite or SQL Server) for persistent storage

At present all data is held in memory and is lost when the application closes. Adding a database such as SQLite (lightweight, file-based, ideal for a desktop app) or SQL Server would persist requests, issues, and events between sessions. Both integrate cleanly with C# through ADO.NET or Entity Framework, making them highly compatible with the existing code. This is the single most valuable enhancement, since a service-tracking system is only useful if data survives restarts.

7.2 Entity Framework Core as an ORM

Rather than writing raw SQL, Entity Framework Core would let the existing C# model classes (ServiceRequest, LocalEvent) map directly to database tables. This reduces boilerplate, lowers the risk of SQL errors, and keeps the data layer in familiar C#. It is the standard data-access technology for .NET applications, so compatibility is excellent.

7.3 Migration to .NET 8 and WPF or MAUI

Windows Forms is mature but dated. Migrating to a modern .NET 8 stack with WPF (richer desktop UI) or .NET MAUI (cross-platform, including mobile) would future-proof the application and allow it to run on phones, which is where many residents would actually access municipal services. The C# logic and data structures would largely carry over, easing the transition.

7.4 A REST API and cloud hosting

Exposing the back-end through an ASP.NET Core Web API hosted on Azure would let multiple clients (desktop, web, mobile) share the same live data, and would allow real municipal staff to update request statuses centrally. This matches the real-world need for many residents and staff to interact with the same system at once.

7.5 Automated testing with xUnit or NUnit

The custom data structures (AVL rotations, red-black balancing, Prim's MST) are exactly the kind of logic that benefits from unit tests. Adding an xUnit or NUnit test project would verify that the trees stay balanced and the algorithms stay correct as the code evolves. These frameworks are the C# standard and integrate directly into Visual Studio.

7.6 Logging with Serilog

For a system used by a municipality, an audit trail of who changed a request and when is important. Serilog is a well-established C# logging library that could record status changes and errors to a file or database with minimal code, improving accountability and making problems easier to diagnose.

8. Updates Made Based on Lecturer Feedback

Parts 1 and 2 of this project were both submitted and marked, and no corrective feedback was received on either submission; both were assessed as strong submissions. As no specific issues were flagged by the lecturer, no part of the application was changed in direct response to feedback.

Although no formal feedback prompted them, a few refinements were made independently between Parts 2 and 3, driven by self-review and testing of the running application. These are documented here for transparency:

- Windows Forms layout issues (overlapping controls and a search results panel that counted matches but failed to display them) were identified through testing and corrected by fixing the order in which docked controls were added to their parent containers, as detailed in Section 5.
- The Service Request Status feature was reviewed against the Task 3 brief and two additional tree types, a Basic Tree and a plain Binary Tree, were added so that all five required tree structures are represented as distinct, clearly-differentiated implementations rather than only the three originally planned (BST, AVL, Red-Black).
- An Admin area with approve/reject functionality was added after the initial Part 3 build to give the application a complete administrative workflow alongside the citizen-facing features.

9. Conclusion

The Municipal Services Application has been completed across all three parts and meets the outlined specifications. The final Part 3 feature, Service Request Status, demonstrates the correct and efficient use of basic trees, binary trees, binary search trees, AVL trees, red-black trees, heaps, graphs, graph traversal, and a minimum spanning tree, each applied to a genuine problem within the system. The project has reinforced the importance of selecting appropriate data structures, understanding the development framework, and structuring code so that it can grow without becoming fragile. With the recommended additions, most importantly persistent storage, the application would be well positioned for real-world municipal use.